

FANTASY FOOTBALL INTELLIGENCE HUB

Software Design Specification

Architecture, Data Dictionary, Analytical Design, and Revision History

Document version	2.0
Application baseline	Fantasy Football Intelligence Hub v2.0
Status	Living design specification
Created	July 26, 2026
Last updated	July 27, 2026
Created by	Mohamad Mokbel
Document purpose	Controlled master specification updated with each application release.

Living-document policy: This specification is the authoritative design record for the application. Each released application change should update the application baseline, relevant architecture or data sections, and the revision history before the new build is distributed.

Document Control

Control	Current value	Maintenance rule
Owner	Mohamad Mokbel	Owner approves scope and release descriptions.
Application version	2.0	Update for every released HTML version.
SDS version	2.0	Normally aligned to the application version.
Architecture status	Current browser architecture plus recommended automation	Update when integrations, hosting, or processing boundaries change.
Data dictionary status	Current through Player Tiers v2.0	Add, change, or deprecate fields whenever the application data model changes.

Table of Contents

1. Executive Summary
2. Scope and Product Objectives
3. System Context and Architecture
4. Application Components and User Experience
5. Data Processing and Analytical Logic
6. External Data Sources and Integration Design
7. Storage, Caching, and Export Design
8. Deployment, Mobile, Security, and Reliability
9. Data Dictionary and Source Lineage
10. Weekly Trend Reporting and Future Data Platform
11. Revision History
12. Release Maintenance Checklist

1. Executive Summary

Fantasy Football Intelligence Hub is a reusable, single-page browser application that analyzes any supported public Sleeper dynasty league. The application retrieves league-specific and player-market data, normalizes identities, calculates legal optimal lineups, ranks teams, evaluates roster risk and draft capital, generates evidence-backed recommendations, and exports reports for offline use.

Version 2.0 adds a Player Tiers page that broadens analysis beyond league rosters to include relevant active free agents when projection or market-value signals are available. Player tiers are calculated primarily from upcoming-season expected PPG, with supporting dynasty-value and career-longevity components.

Current processing boundary: The released application performs API retrieval, normalization, analytics, caching, and rendering inside the user's browser. nflverse integration and scheduled historical snapshot automation remain recommended enhancements rather than current released functionality.

2. Scope and Product Objectives

2.1 In Scope

- Analyze any public Sleeper NFL league using a league ID.
- Detect league format, scoring, roster slots, and linked prior seasons.
- Aggregate current and prior Sleeper matchup production.
- Combine projections and dynasty values from supported public sources.
- Calculate legal optimal lineups, depth, risk, contender score, franchise score, and draft-capital strength.
- Provide team insights, trade targets, player master data, player tiers, and exportable reports.
- Operate as a portable single HTML file with local browser caching.

2.2 Out of Scope in the Current Release

- Authenticated private-league access.
- Automated KeepTradeCut data collection or scraping.
- Licensed proprietary PFF or commercial real-time feeds.
- Guaranteed same-origin backend processing.
- Automatic weekly historical snapshots across all users.
- Completed direct nflverse browser integration.

2.3 Primary Users

- Dynasty league managers evaluating their own roster.
- Commissioners reviewing competitive balance and draft capital.
- Analysts creating weekly player, team, and market reports.
- Mobile and desktop users accessing a hosted version of the HTML application.

3. System Context and Architecture

Fantasy Football Intelligence Hub — Application Architecture & Data Flow

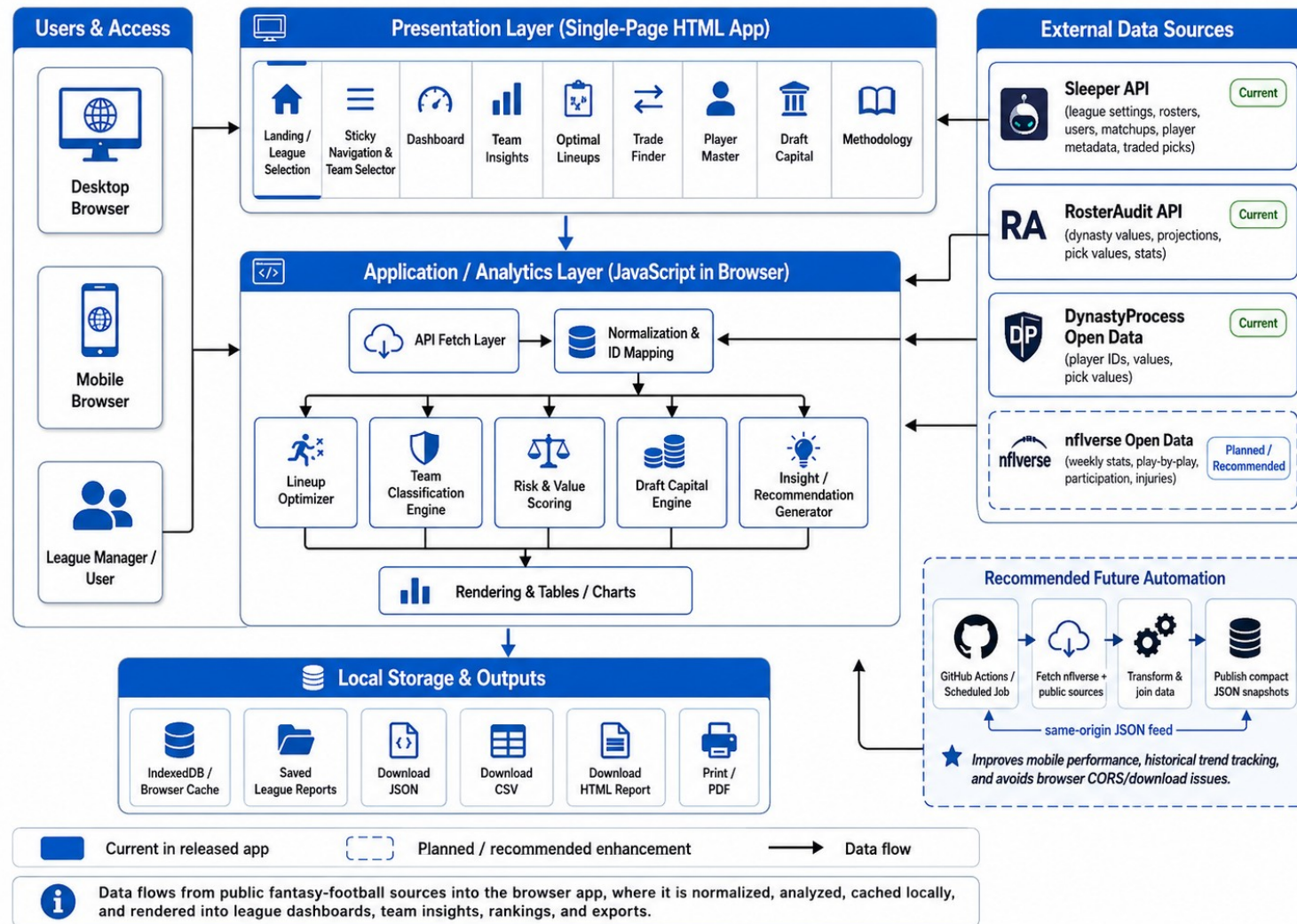


Figure 1. Fantasy Football Intelligence Hub application architecture and data flow.

3.1 Current Architecture

- Presentation layer: landing page, sticky navigation, league dashboard, team insights, optimal lineups, trade finder, player master, player tiers, draft capital, and methodology.
- Application layer: JavaScript fetch services, normalization and player-ID mapping, lineup optimization, ranking engines, risk/value scoring, pick ownership, tier scoring, and insight generation.
- External sources: Sleeper API, RosterAudit API, and DynastyProcess open files.
- Local persistence: IndexedDB daily player cache, saved analysis by league ID, and downloadable JSON/CSV/HTML outputs.
- Recommended future layer: scheduled GitHub Actions or similar free automation that publishes compact same-origin snapshots, including nflverse data.

3.2 End-to-End Data Flow

Stage	Processing description
1. League selection	The user enters a Sleeper league ID and optional format/history settings.
2. Source retrieval	The browser retrieves league, user, roster, matchup, player, traded-pick, projection, and value data.
3. Normalization	Player identities are joined primarily with Sleeper IDs and supplemented with cross-platform mappings and normalized names.
4. Analytics	The application builds legal lineups, team metrics, player risk, positional ranks, player tiers, draft capital, and recommendations.
5. Rendering	The UI renders league-wide and focus-team views, tables, charts, cards, evidence, and filters.
6. Persistence and export	The report is cached locally and can be downloaded in JSON, CSV, embedded HTML, or print/PDF form.

4. Application Components and User Experience

Component	Responsibility
Landing / League Selection	Collects league ID, format override, prior-season depth, future-pick horizon, and ranking weights. Runs analysis or loads a saved report.
Persistent Header and Navigation	Displays logo, focus-team selector, version control, creator/date information, tabs, league metadata, and refresh/change-league actions.
Dashboard	Shows league rankings, projected lineup PPG, contender classification, focus-team KPIs, and evidence-backed recommendations.
Team Insights	Shows strengths, weaknesses, build-around assets, sell/reassess candidates, optimal lineup, tiers, risk, depth, and draft-capital context.
Optimal Lineups	Uses a constrained assignment algorithm to place eligible non-taxi players into the league's actual starting slots.
Trade Finder	Identifies projected upgrades relative to the focus team's weakest legal starting slot and ranks targets by gain relative to market

Component	Responsibility
	cost.
Player Master	Provides filterable rostered and relevant free-agent player data including actual production, projection, value, risk, identity, and source information.
Player Tiers	Classifies relevant QB/RB/WR/TE players using 60% projection, 25% dynasty value, and 15% longevity; supports position, tier, roster status, and sort filters.
Draft Capital	Displays capital leaderboard, team portfolios, original ownership, current ownership, previous owner when available, and acquired/sent movement.
Methodology	Explains formulas, classification rules, sources, limitations, and tier interpretation.
Exports	Downloads analysis JSON, ranking CSV, player CSV, embedded HTML report, or print/PDF output.

5. Data Processing and Analytical Logic

5.1 Identity and Normalization

- Sleeper player ID is the primary key for current application joins.
- DynastyProcess mappings supply GSIS, FantasyPros, KeepTradeCut, and other identifiers where available.
- Normalized player names are used only as a fallback match after punctuation, suffix, and accent cleanup.
- Numeric and null-conversion helpers standardize inconsistent JSON/CSV source values.
- Roster status is assigned from Sleeper starter, taxi, reserve, and player collections.

5.2 Legal Lineup Optimization

The application models starting slots and eligible players as a minimum-cost flow problem. Player expected PPG is converted to a negative cost so the solver maximizes total expected PPG while assigning each player to at most one eligible slot. Taxi players are excluded from legal lineup selection.

5.3 Contender and Franchise Scores

Metric	Default contender weight	Role
Projected legal lineup	40%	Primary current-season strength signal.
Depth	15%	Bench and injury insulation.
Production	15%	Current or most recent completed-season points and maximum points.
Dynasty value	15%	Market-supported roster quality.
Risk	10%	Age, injury, role, projected games, and fragility.
Draft capital	5%	Future strategic flexibility.

5.4 Player Tier Model

Tier	Composite range	Meaning	Primary interpretation
------	-----------------	---------	------------------------

S	92-100	Elite cornerstones	Elite projection plus market and longevity support.
1	80-91.9	High-end starters	Strong weekly projection and durable dynasty profile.
2	65-79.9	Strong starters / core assets	Reliable contributors with at least two supporting dimensions.
3	45-64.9	Useful starters / depth	Usable production with projection, value, or longevity limitations.
4	Below 45	Speculative / replacement level	Weak current signal or material role/career risk.

Player tier composite = 60% same-position expected-PPG percentile + 25% same-position dynasty-value percentile + 15% same-position longevity percentile. Longevity applies position-specific age curves and adjusts for injury, depth role, NFL team status, and the existing risk score.

6. External Data Sources and Integration Design

Source	Status	Access method	Primary contribution	Key limitation
Sleeper	Current	Public JSON API	League configuration, rosters, managers, matchups, players, scoring, and traded picks.	Local file access and cross-origin behavior can vary by browser.
RosterAudit	Current	Public JSON API	Dynasty values, trends, projections, player/pick information.	Availability and CORS remain external dependencies; attribution required.
DynastyProcess	Current	Open CSV files	Secondary values and cross-platform player identifiers.	Static file schemas and upstream refresh cadence may change.
nflverse	Planned / recommended	Open release assets or scheduled preprocessing	Weekly NFL statistics, play-by-play, participation, injuries, and advanced derived usage.	Direct GitHub release fetch previously failed in browser; use scheduled same-origin snapshots.

6.1 nflverse Failure and Recommended Resolution

The prior NFLverse attempt recorded only “Failed to fetch,” without an HTTP status or detailed browser exception. The likely failure boundary was the browser request to a GitHub release asset that redirects to a signed release-assets URL. This can fail because of CORS, file:// origin restrictions, redirect handling, attachment delivery, privacy controls, or time-limited asset URLs.

- Use a free scheduled GitHub Actions workflow to fetch nflverse data outside the browser.
- Convert only required weekly fields into compact JSON.
- Join GSIS and Sleeper identifiers during the scheduled job.
- Publish the output beside the hosted application so the browser reads a same-origin feed.
- Retain dated snapshots to enable true weekly deltas and historical trend reports.

7. Storage, Caching, and Export Design

- IndexedDB stores the large Sleeper player directory with a daily cache key.
- Completed analysis is cached by league ID for later reopening without a full refresh.
- The embedded-report export stores the current analysis JSON inside a copy of the HTML application.
- CSV exports flatten team rankings and player-master data for spreadsheet analysis.
- No central database currently exists; browser caches are user/device specific and may be cleared by browser storage policies.

8. Deployment, Mobile, Security, and Reliability

8.1 Deployment

The application can remain a single HTML file, but reliable API access requires hosting over HTTPS. Opening the application directly through a file:// URL is less reliable because mobile and desktop browsers may restrict cross-origin requests. GitHub Pages, Netlify, Vercel, or another static HTTPS host is suitable for the current design.

8.2 Mobile Design

- Mobile browsers can execute JavaScript fetch requests when the page is hosted over HTTP/HTTPS and each source permits CORS.
- Cards should collapse to one column, controls should use touch-sized targets, and tables/navigation should scroll horizontally rather than overlap.
- Large data sources should be preprocessed into compact same-origin JSON to improve mobile speed and reliability.
- IndexedDB is supported on modern mobile browsers, but iOS may purge storage under device-pressure conditions.

8.3 Security and Privacy

- No Sleeper login credentials are collected; only public league IDs are used.
- The current application does not transmit analysis to an application-owned server.
- All external calls should use HTTPS.
- Exported reports can contain manager display names, team names, league IDs, and roster details and should be handled as user-controlled files.
- Future hosted automation should use least-privilege repository tokens and must not expose secrets in client-side code.

8.4 Reliability and Observability

- Use Promise.allSettled for optional sources so one provider does not prevent Sleeper analysis.
- Display per-source status badges and retain fetch-error details.
- Cache stable identity data and provide Force Refresh for time-sensitive data.
- Validate duplicate HTML IDs, CSS parsing, JavaScript syntax, and responsive width calculations before release.
- For scheduled data jobs, retain job logs, source timestamps, row counts, schema version, and checksum.

9. Data Dictionary and Source Lineage

This section incorporates the standalone data dictionary and updates its application baseline to Version 2.0. Source terminology and field definitions are retained from the original working document, with an additional Player Tiers extension.

Fantasy Football Intelligence Hub

Working Draft 1.0 | Created July 26, 2026 | Mohamad Mokbel

Purpose	Inventory every data point currently collected or calculated, document source lineage, and identify weekly trend-report opportunities.
Primary application reviewed	Fantasy Football Intelligence Hub v2.0 browser application.
Supplemental datasets reviewed	Ordinary Gentlemen final master dataset and multi-season Sleeper league export.
Scope distinction	Fields are labeled as Current App, Supplemental Pipeline, Raw Source, or Derived Metric.

1. Data Architecture Overview

The system currently operates as an in-browser analytical pipeline. It retrieves public data from Sleeper, RosterAudit, and DynastyProcess, normalizes player identities, calculates production and projection metrics, solves legal starting lineups, calculates team scores, and caches the completed analysis in IndexedDB. A previous master-data pipeline also used NFLverse weekly statistics and contains several fields not yet exposed by the browser application.

Layer	Purpose	Examples	Persistence
Source APIs / files	Raw information obtained from external systems.	Sleeper rosters, matchups, players and traded picks; RosterAudit values and projections; DynastyProcess IDs and values; NFLverse weekly stats.	Fetches live or loaded from static CSV/JSON.
Normalization	Standardizes IDs, names, nulls, formats and ownership relationships.	Sleeper ID joins, name normalization, numeric conversion, roster-status assignment.	Held in browser memory during a run.
Player analytical model	One normalized record per rostered player.	Expected PPG, consensus value, risk score, position tier.	Stored in analysis JSON and browser cache.
Team analytical model	One record per fantasy roster.	Optimal lineup, depth index, contender score, franchise score, evidence-backed insights.	Stored in analysis JSON and browser cache.
Pick ownership model	One record per future pick.	Original team, current owner, prior owner, year, round and model value.	Stored in analysis JSON and browser cache.
Presentation / exports	Dashboards and downloadable outputs.	HTML report, analysis JSON, rankings CSV, player CSV, print/PDF.	User-controlled download or browser cache.

1.1 Data Grain

Dataset / object	Grain	Natural key
League analysis	One record per analysis run.	leagueId + generatedAt
League history	One record per linked Sleeper season.	leagueId / season
Player master	One record per rostered player per analysis run.	leagueId + rosterId + sleeperId + generatedAt
Player weekly production (raw)	One record per player per matchup week.	leagueId + week + sleeperId
Team analytical model	One record per roster per analysis run.	leagueId + rosterId + generatedAt
Optimal lineup assignment	One record per starting slot per roster per run.	leagueId + rosterId + slot index + generatedAt
Future pick ownership	One record per season, round and original roster.	leagueId + season + round + originRosterId
Transaction (available in export)	One record per Sleeper transaction.	transaction_id
Draft (available in export)	One record per Sleeper draft.	draft_id when retained by source

1.2 Refresh and Historical Behavior

- Sleeper league, roster, matchup, player, and traded-pick information is refreshed when Analyze League or Force Refresh is run.
- The large Sleeper player directory is cached daily in IndexedDB.
- Completed analysis is cached by league ID and can be reopened without calling APIs.
- The current application does not automatically retain a dated weekly snapshot history. Therefore, trend7d and trend30d can be displayed when provided by RosterAudit, but most custom week-over-week trend reports require saving one analysis snapshot per week.
- The multi-season Sleeper export contains prior-season rosters, matchups, brackets, drafts and limited transactions and can support historical league reporting independently of the browser cache.

2. Source System Catalog

Source	Coverage	Refresh	Endpoint / asset	Data supplied	Notes
Sleeper League API	Current App	Live	/league/{id}	League name, season, status, settings, scoring, roster positions, linked prior league.	Public read-only API; core league configuration.
Sleeper Users API	Current App	Live	/league/{id}/users	Manager identity, display name, team metadata and avatar.	Team name may come from user metadata or display name.
Sleeper Rosters API	Current App	Live	/league/{id}/rosters	Roster ownership, players, starters, taxi, reserve,	Primary source for fantasy-team membership.

Source	Coverage	Refresh	Endpoint / asset	Data supplied	Notes
				wins/losses, PF and max PF.	
Sleeper Matchups API	Current App	Weekly	/league/{id}/matchups/{week}	Player points, roster points, starters and matchup IDs.	Used to reconstruct actual player and team production.
Sleeper Traded Picks API	Current App	Live	/league/{id}/traded_picks	Original roster, current owner, previous owner, season and round.	Used for pick ownership and movement visualization.
Sleeper Drafts API	Current App / Raw	Event based	/league/{id}/drafts	Draft metadata and, in the export, picks and traded picks.	Retrieved by app but current dashboards primarily use future traded picks.
Sleeper NFL State API	Current App	Live	/state/nfl	Current NFL season, season type and week/leg.	Determines matchup weeks to fetch.
Sleeper Player Directory	Current App	Daily cache	/players/nfl	Player name, position, NFL team, age, status, injury and depth chart.	Large response; cached once per day.
Sleeper Projections	Current App	As published	api.sleeper.com/projections/nfl/{season}	Projected PPR points and games when available.	Fallback when RosterAudit projection is missing.
RosterAudit Values	Current App	As published	/rankings/values?format_key=...	1QB/SF values plus 7-day and 30-day market trends when present.	Primary market-value source; attribution required.
RosterAudit Projections	Current App	As published	/projections/ppg-rankings	Projected PPG, games, dynasty value and projection stat payload.	Primary projection source.
RosterAudit Picks	Fetches / partially used	As published	/picks	Published future-pick value data.	Current team pick scoring uses internal fallback round values rather than this payload.
DynastyProcess Player IDs	Current App	Repository update	db_playerids.csv	Sleeper, GSIS, FantasyPros and KTC IDs; draft and college fields.	Cross-platform identity bridge.
DynastyProcess Values	Current App	Repository update	values.csv	1QB and 2QB dynasty values and age/name metadata.	Secondary market-value source.
NFLverse Weekly Stats	Supplemental Pipeline	Weekly/season file	stats_player_week_{year}.csv	Weekly NFL box-score statistics used to calculate custom fantasy points.	Not currently called by the browser app; present in final master dataset methodology.

Source	Coverage	Refresh	Endpoint / asset	Data supplied	Notes
Sleeper Multi-season Export	Supplemental File	Snapshot	Local JSON export	League/users/ rosters/matchups/ drafts/brackets/ traded picks/transactions for linked seasons.	Supports historical audits and custom longitudinal reports.

3. League and Run-Level Data Dictionary

Field	Type	Source class	Raw source / formula	Definition	Use
appVersion	string	Derived / application	Application constant	Version of the HTML application that generated the analysis.	Audit / reproducibility.
generatedAt	datetime	Derived / application	Browser timestamp	Exact date and time the analysis was created.	Required for future weekly snapshots.
leagueId	string	Sleeper league	league.league_id	Sleeper league identifier entered by the user.	Primary league key.
leagueName	string	Sleeper league	league.name	League display name.	Header and exports.
season	integer	Sleeper league	league.season	Fantasy season represented by the selected league.	Season partition.
leagueStatus	string	Sleeper league	league.status	Pre-draft, drafting, in season, or complete status.	Controls matchup loading logic.
currentWeek	integer	Sleeper NFL state + logic	state.leg / state.week	Highest matchup week fetched for the current season.	Zero in preseason.
totalRosters	integer	Sleeper rosters	rosters.length	Number of fantasy teams.	Ranking denominator.
formatKey	string	Derived	detectFormat()	Internal format code such as 1qb_ppr or sf_half.	Chooses market-value source fields.
formatLabel	string	Derived	scoringFormatLabel()	Human-readable league format.	Presentation.
rosterSlots	array<string>	Sleeper league + derived	league.roster_positions	Legal starting slots after BN, IR and TAXI are excluded.	Input to lineup optimizer.
unsupportedSlots	array<string>	Derived	IDP slot filter	IDP positions lacking complete public market-value coverage.	Warning mechanism.

Field	Type	Source class	Raw source / formula	Definition	Use
weights.projection	decimal	User configuration	Advanced scoring controls	Normalized contender-score weight for optimal lineup projection.	Default 40%.
weights.depth	decimal	User configuration	Advanced scoring controls	Normalized contender-score weight for bench depth.	Default 15%.
weights.history	decimal	User configuration	Advanced scoring controls	Normalized contender-score weight for production.	Default 15%.
weights.dynasty	decimal	User configuration	Advanced scoring controls	Normalized contender-score weight for dynasty value.	Default 15%.
weights.risk	decimal	User configuration	Advanced scoring controls	Normalized contender-score weight for low risk.	Default 10%.
weights.picks	decimal	User configuration	Advanced scoring controls	Normalized contender-score weight for draft capital.	Default 5%.
useCurrentProduction	boolean	Derived	Current week ≥ 4 and current PF exists	Whether current-season team production replaces prior-season production in contender score.	Avoids overreacting to tiny early samples.
sourceStatuses	array<object>	Derived / runtime	API results	Status, detail and availability of each data source.	Data-quality transparency.
history	array<object>	Sleeper linked leagues	previous_league_id traversal	League ID, season, name and status for linked seasons.	Historical navigation.
methodology	object	Application	Static descriptions	Human-readable formulas and source use.	Export transparency.

4. Raw Sleeper Structures Available

These fields exist in the source/export even when the current dashboard does not directly expose all of them. They are important for future trend and operational reports.

Object	Fields currently observed in export	Notes
League object	league_id, name, season, season_type, status, sport, total_rosters,	Raw source availability; field shape can vary by league state and Sleeper response.

Object	Fields currently observed in export	Notes
	previous_league_id, roster_positions, scoring_settings, settings, metadata, draft_id, avatar, bracket_id, loser_bracket_id	
User object	user_id, display_name, avatar, metadata.team_name, metadata, settings, is_owner, is_bot, league_id	Raw source availability; field shape can vary by league state and Sleeper response.
Roster object	roster_id, owner_id, co_owners, players, starters, taxi, reserve, keepers, player_map, settings.wins, settings.losses, settings.ties, settings.fpts, settings.fpts_decimal, settings.ppts, settings.ppts_decimal, metadata	Raw source availability; field shape can vary by league state and Sleeper response.
Matchup object	week (container key), matchup_id, roster_id, points, custom_points, players, players_points, starters, starters_points	Raw source availability; field shape can vary by league state and Sleeper response.
Traded pick object	season, round, roster_id (original team), owner_id (current owner), previous_owner_id	Raw source availability; field shape can vary by league state and Sleeper response.
Draft object	details, picks, traded_picks	Raw source availability; field shape can vary by league state and Sleeper response.
Transaction object	transaction_id, type, status, created, status_updated, leg, creator, roster_ids, consenter_ids, adds, drops, draft_picks, waiver_budget, settings, metadata	Raw source availability; field shape can vary by league state and Sleeper response.
Bracket object	winners_bracket, losers_bracket and each source-provided node fields	Raw source availability; field shape can vary by league state and Sleeper response.

5. Current Application Player Data Dictionary

Field	Type	Source	Raw field / formula	Definition	Primary use
rosterId	integer	Sleeper roster	roster.roster_id	Fantasy roster currently holding the player.	Team joins.
fantasyTeam	string	Derived from Sleeper user/roster	teamMap	Fantasy team display name.	Display and filters.
manager	string	Sleeper user	user.display_name	Manager display name.	Ownership context.
sleeperId	string	Sleeper roster/player directory	player ID	Primary player key in current app.	Joins across sources.
name	string	Sleeper / DynastyProcess fallback	full_name or normalized name	Player display name.	Presentation and fallback matching.
position	string	Sleeper / DynastyProcess	position	Player position.	Eligibility, tiers and reports.

Field	Type	Source	Raw field / formula	Definition	Primary use
nflTeam	string	Sleeper / DynastyProcess	team	Current NFL team abbreviation.	Team context and FA detection.
age	decimal/integer	Sleeper / DynastyProcess	age	Current player age.	Age curve and youth metrics.
status	string	Sleeper player directory	status	NFL roster/activity status.	Risk.
injuryStatus	string	Sleeper player directory	injury_status	Current injury designation.	Risk and weekly availability.
depthOrder	integer	Sleeper player directory	depth_chart_order	Published depth chart order.	Risk and role context.
rosterStatus	string	Derived from Sleeper roster arrays	Starter / Bench / Reserve / Taxi	Fantasy roster location at analysis time.	Lineup eligibility and filtering.
actualPoints	decimal	Sleeper matchups	sum(players_points[player])	Fantasy points accumulated in fetched matchup weeks.	Current production.
actualPpg	decimal	Derived	actualPoints / scoringWeeks	Average over weeks with non-zero fantasy points.	Production rate.
rosteredWeeks	integer	Derived from matchups	count of matchup entries	Weeks observed on a roster in fetched data.	Availability/sample size.
scoringWeeks	integer	Derived from matchups	count(points != 0)	Weeks with non-zero fantasy points.	Actual PPG denominator.
starts	integer	Derived from matchup starters	count starter appearances	Fantasy starts during fetched weeks.	Usage by fantasy manager.
starterPpg	decimal	Derived	starterPoints / starts	Average points in fantasy starts.	Start/sit effectiveness.
projectedTotal	decimal	RosterAudit or Sleeper or fallback	RA PPG × games; else Sleeper total; else actualPpg × 17	Normalized projected season total.	Optimal lineup.
projectedGames	decimal/integer	RosterAudit / Sleeper	projection games	Projected games available.	Risk and projection context.
expectedPpg	decimal	Derived	projectedTotal / 17	Availability-adjusted weekly expectation.	Primary optimizer metric.
sourcePpg	decimal	RosterAudit / Sleeper / actual	raw source PPG	Unadjusted projection PPG when available.	Reference and debugging.
projectionSource	string	Derived	RosterAudit / Sleeper / Actual fallback	Source selected for projection.	Data lineage.
dynastyValueRA	decimal	RosterAudit	1qb or sf field	RosterAudit market value.	Consensus input.
dynastyValueDP	decimal	DynastyProcess	value_1qb or value_2qb	DynastyProcess market value.	Consensus input.

Field	Type	Source	Raw field / formula	Definition	Primary use
dynastyValue	integer	Derived	mean of available RA and DP values	Consensus market value.	Team value and trade finder.
trend7d	decimal	RosterAudit	trend_7d	Published seven-day market-value trend when returned.	Weekly trend reports.
trend30d	decimal	RosterAudit	trend_30d	Published thirty-day market-value trend when returned.	Monthly momentum.
draftYear	integer	DynastyProcess IDs	draft_year	NFL draft year.	Career cohort.
draftRound	integer	DynastyProcess IDs	draft_round	NFL draft round.	Pedigree.
college	string	Sleeper / DynastyProcess	college	College attended.	Player profile/cohort reports.
gsisId	string	DynastyProcess IDs	gsis_id	NFL data ecosystem identifier.	Future NFLverse joins.
fantasyProsId	string	DynastyProcess IDs	fantasypros_id	FantasyPros identifier.	External joins.
ktcId	string	DynastyProcess IDs	ktc_id	KeepTradeCut identifier.	Potential future joins.
riskScore	integer 0-100	Derived	riskFor()	Composite age, injury, status, depth, projected games and value fragility score.	Risk ranking.
riskTier	string	Derived	Low / Moderate / High / Very High	Categorical risk label.	Presentation.
reasons	array<string>	Derived	riskFor() flags	Human-readable factors behind risk score.	Evidence-backed insights.
positionRank	integer	Derived	rank by expectedPpg within position	Player rank among rostered league players at same position.	Tier/evidence.
positionPoolSize	integer	Derived	count at position	Number of rostered players in positional ranking pool.	Rank context.
positionPercentile	decimal	Derived	relative rank percentile	Expected-PPG percentile at position.	Tier assignment.
positionTier	string	Derived	Tier 1-4 from percentile thresholds	Tier 1 top 10%; Tier 2 next 15%; Tier 3 next 25%; Tier 4 remainder.	Trend and roster evidence.

6. Supplemental Master-Dataset Player Fields

The final master dataset contains additional identity, detailed projection and reconstructed historical fields. These are available in the supporting JSON but are not all generated or displayed by the current browser application.

Field	Source / derivation	Definition	Potential weekly use
birthdate	Sleeper / identity enrichment	Player date of birth.	Age curves and birthday-based age calculation.
years_exp	Sleeper	NFL experience.	Breakout and career-stage cohorts.
draft_pick	Identity mapping	Overall draft selection.	Draft-capital pedigree.
player_status	Sleeper	Player activity/status field.	Availability trend.
points_2025_custom	NFLverse + league scoring	Custom-scored 2025 total.	Historical baseline.
games_2025	NFLverse	Games represented in 2025 weekly stats.	Sample size.
ppg_2025_custom	Derived	Custom points / games.	Historical rate.
sleeper_export_points_2025	Sleeper export	Sleeper-observed 2025 points.	Reconciliation.
proj_2026_games	RosterAudit	Projected games.	Availability.
proj_2026_ppg_ppr	RosterAudit	Projected PPR PPG.	Projection reporting.
proj_2026_total_ppr	Derived	Projected PPG × games.	Season total.
proj_2026_pass_yd	RosterAudit projection stats	Projected passing yards.	QB trend/reporting.
proj_2026_pass_td	RosterAudit projection stats	Projected passing touchdowns.	QB trend/reporting.
proj_2026_rush_yd	RosterAudit projection stats	Projected rushing yards.	QB/RB trend reporting.
proj_2026_rush_td	RosterAudit projection stats	Projected rushing touchdowns.	Scoring upside.
proj_2026_rec	RosterAudit projection stats	Projected receptions.	PPR volume.
proj_2026_rec_yd	RosterAudit projection stats	Projected receiving yards.	Receiving volume.
proj_2026_rec_td	RosterAudit projection stats	Projected receiving touchdowns.	Scoring upside.
dynasty_value_rosteraudit_1qb	RosterAudit	1QB value.	Market source detail.
dynasty_value_dynastyprocess_1qb	DynastyProcess	1QB value.	Market source detail.
dynasty_value_consensus	Derived	Mean of available market values.	Consensus.
dynasty_ecr_1qb	RosterAudit / supplemental	Expert consensus rank if present.	Market rank reporting.
dynasty_value_date	Source metadata	Date of value snapshot.	Historical trend lineage.
points_2025_sleeper	Sleeper matchups	Reconstructed 2025 points.	Historical baseline.
weeks_rostered_2025	Sleeper matchups	Weeks observed on roster.	Roster continuity.
weeks_scored_2025	Sleeper matchups	Weeks with nonzero points.	PPG denominator.
ppg_2025_scoring_weeks	Derived	Points / scoring weeks.	Active-week production.
ppg_2025_rostered_weeks	Derived	Points / rostered weeks.	Rostered-week efficiency.
starts_2025	Sleeper matchups	Fantasy starts.	Manager usage.

Field	Source / derivation	Definition	Potential weekly use
starter_points_2025	Sleeper matchups	Points scored while started.	Start contribution.
starter_ppg_2025	Derived	Starter points / starts.	Start efficiency.
proj_2026_sleeper_total_ppr	Sleeper projections	Sleeper projected total.	Fallback/audit.
proj_2026_sleeper_games	Sleeper projections	Sleeper projected games.	Fallback/audit.
proj_2026_sleeper_ppg_ppr	Derived	Sleeper total / games or supplied PPG.	Fallback/audit.
proj_2026_sleeper_source	Derived	Sleeper projection endpoint/field.	Lineage.
projection_2026_primary_source	Derived	Chosen projection source.	Lineage.

7. Team Analytical Data Dictionary

Field	Type	Definition	Use
rosterId	integer	Sleeper roster ID.	Team key.
team	string	Fantasy team name.	Display.
manager	string	Manager display name.	Display.
players	array<player>	All normalized players on roster.	Roster analysis.
lineup	array<assignment>	Optimal legal slot-to-player assignments.	Projected lineup.
bench	array<player>	Players not selected by optimizer, excluding taxi.	Depth.
positionScores	object	Sum of expected PPG by projected positional group.	Positional strength evidence.
lineupPpg	decimal	Sum of expectedPpg for optimal lineup.	Primary projection.
lineupSourcePpg	decimal	Sum of unadjusted source PPG.	Reference.
depth	decimal	45% of expected PPG from top six eligible bench players.	Bench index.
risk	decimal	Expected-PPG weighted risk score of optimal lineup.	Team risk.
lineupAge	decimal	Expected-PPG weighted age of optimal lineup.	Age profile.
highRiskStarters	integer	Count of projected starters with riskScore >= 40.	Risk evidence.
totalValue	integer	Sum of player consensus values.	Franchise value.
lineupValue	integer	Sum of consensus values in optimal lineup.	Starting asset value.

Field	Type	Definition	Use
youngValue	integer	Value of RB/WR/TE players under age 27.	Long-term core.
currentWins	integer	Current roster wins.	Production record.
currentLosses	integer	Current roster losses.	Production record.
currentPF	decimal	Current-season points for.	Production metric.
currentMaxPF	decimal	Current-season maximum points / potential points.	Roster strength and lineup efficiency.
priorWins	integer	Prior linked season wins.	Historical baseline.
priorLosses	integer	Prior linked season losses.	Historical baseline.
priorPF	decimal	Prior linked season points for.	Historical production.
priorMaxPF	decimal	Prior linked season maximum points.	Historical roster quality.
pickCapital	integer	Sum of internal model values of owned future picks.	Capital score.
futureFirsts	integer	Count of owned first-round future picks.	Capital summary.
picksOwned	array<pick>	Detailed owned future picks.	Ownership visualization.
projectionPct	decimal	Percentile of lineupPpg across league.	Contender score.
depthPct	decimal	Percentile of depth.	Contender score.
productionPct	decimal	Percentile of production metric.	Contender score.
dynastyPct	decimal	Percentile of totalValue.	Contender/franchise score.
riskPct	decimal	Inverse percentile of risk.	Contender/franchise score.
picksPct	decimal	Percentile of pickCapital.	Contender/franchise score.
youngPct	decimal	Percentile of youngValue.	Franchise score.
contenderScore	decimal	Weighted percentile composite.	Current power ranking.
franchiseScore	decimal	40% dynasty + 25% youth + 15% picks + 10% projection + 10% low risk.	Long-term ranking.
currentRank	integer	Descending contenderScore rank.	League power rank.
franchiseRank	integer	Descending franchiseScore rank.	Long-term rank.
currentClass	string	Favorite / Strong / Fringe / Middle / Rebuild classification.	Strategy.
franchiseClass	string	Elite / Strong & Sustainable / Mixed / Rebuild Required.	Long-term strategy.
insights	object	Evidence-backed strengths, weaknesses, strategy, build-around and shop candidates.	Narrative output.

8. Lineup Assignment and Draft Pick Objects

8.1 Optimal Lineup Assignment

Field	Type	Definition
slot	string	Starting slot from league roster configuration, including FLEX or SUPER_FLEX.
player	player object	Reference to the selected normalized player record.
Eligibility rule	derived	Exact position match or configured flex eligibility.
Selection method	derived	Min-cost/max-value flow optimization maximizing total expectedPpg without assigning one player twice.
Taxi handling	derived	Taxi players are excluded from legal lineup candidates.

8.2 Future Pick Ownership

Field	Type	Definition
season	integer	Future draft year.
round	integer	Draft round.
originRosterId	integer	Roster whose original pick it is.
ownerRosterId	integer	Current owner.
previousOwnerRosterId	integer	Previous owner reported by Sleeper.
originTeam	string	Original fantasy team name.
ownerTeam	string	Current owner team name.
previousOwnerTeam	string	Previous owner team name.
acquired	boolean	True when current owner differs from original roster.
traded	boolean	True when pick has moved.
value	integer	Internal fallback model value by round: R1 4000, R2 1800, R3 800, R4 350, lower rounds declining.

9. Calculation and Classification Dictionary

Metric	Formula / rule	Interpretation
Expected PPG	$\text{projectedTotal} / 17$	Availability-adjusted projection used by optimizer. A player projected for fewer games is not treated as if available all season.
Consensus dynasty value	$\text{mean}(\text{available RosterAudit}, \text{DynastyProcess values})$	Averages available market sources; zero when neither is available.
Depth index	$0.45 \times \text{sum}(\text{expectedPpg of top 6 bench players})$	Current browser formula. This is not a direct projected lineup replacement score.
Team risk	weighted average of starter riskScore using expectedPpg weights	Higher-production starters influence team risk more heavily.
Lineup age	weighted average of starter ages using expectedPpg weights	Describes projected lineup age rather than full roster age.
Position score	sum expectedPpg of optimal starters at position/group	Used to rank positional units and support narrative claims.
Production metric	$(\text{PF} + \text{max PF}) / 2$	Uses current season after Week 4; otherwise prior linked season.
Contender score	weighted percentiles	Defaults: projection 40%, depth 15%, production 15%, dynasty 15%, low risk 10%, picks 5%.
Franchise score	$0.40 \text{ dynastyPct} + 0.25 \text{ youngPct} + 0.15 \text{ picksPct} + 0.10 \text{ projectionPct} + 0.10 \text{ riskPct}$	Long-term sustainability score.
Position tier	rank percentile within rostered league players at same position	Tier 1 top 10%; Tier 2 next 15%; Tier 3 next 25%; Tier 4 remaining 50%.
Risk score	rule-based 0-100	Age curve by position, injury designation, inactive status, free-agent status, depth order, projected games and production/value fragility.
Current classification	rank / league size thresholds	Top 25% Favorite; through 45% Strong; through 65% Fringe; through 82% Middle; remainder Rebuild.
Franchise classification	rank / league size quartiles	Top 25% Elite; top 50% Strong & Sustainable; top 75% Mixed / Retool; remainder Rebuild Required.

10. Weekly Trend Reporting Readiness

The table below distinguishes reports that can be generated immediately from one current snapshot from reports that require retaining dated weekly snapshots or adding a new source.

Report concept	Readiness	Required fields	What it would show
Weekly market movers	Ready now, if RosterAudit returns trend fields	trend7d, trend30d, dynastyValue, position, age, fantasyTeam	Top risers/fallers; roster-level value gained/lost.

Report concept	Readiness	Required fields	What it would show
Projection movers	Requires weekly snapshots	expectedPpg, projectedTotal, projectedGames, projectionSource	Week-over-week projection change; role and availability shifts.
Roster value movement	Requires weekly snapshots	dynastyValue by player and totalValue by team	Team value gained/lost from market changes, adds, drops and trades.
Risk movement	Requires weekly snapshots	riskScore, riskTier, reasons, injuryStatus, depthOrder	Players moving into/out of high-risk categories.
Depth-chart movers	Requires weekly snapshots	depthOrder, nflTeam, status	Players gaining or losing depth-chart position.
Injury watch	Ready for current status; history requires snapshots	injuryStatus, status, projectedGames, risk reasons	Current availability dashboard; new injury and recovery deltas once snapshotted.
Actual scoring trend	Partially ready	weekly Sleeper players_points exists during fetch but only aggregate is retained in final analysis	Rolling 3-week PPG requires retaining weekly player rows in exported analysis.
Starter utilization trend	Partially ready	starters and starts aggregate	Start rate and bench-point decisions require weekly assignment retention.
Lineup efficiency trend	Team-level ready by season; weekly requires additional retention	team points, starter points, max PF	Weekly actual vs optimal/potential lineup.
Position group momentum	Requires weekly snapshots	positionScores, positional rank, player tiers	Which teams improved or declined at QB/RB/WR/TE.
Contender rank movement	Requires weekly snapshots	contenderScore, currentRank, component percentiles	Weekly power-ranking changes and cause decomposition.
Franchise rank movement	Requires weekly snapshots	franchiseScore, totalValue, youngValue, pickCapital	Long-term ranking changes.
Trade impact report	Requires transaction persistence and before/after snapshots	Sleeper transactions, roster ownership, values and projections	Immediate and later value/PPG effect of trades.
Waiver impact report	Requires transaction persistence	adds, drops, waiver budget, player production/value	Best and worst waiver moves.
Pick-value movement	Requires published pick-value history or model changes	pick ownership, origin, round, team ranking	How likely early picks change as original teams rise/fall.
Breakout candidates	Can be designed now; stronger with NFLverse weekly stats	age, experience, draft pedigree, depth, projections, actual weekly stats	Young players with role and production acceleration.
Buy-low / sell-high	Partially ready; best with snapshots	market trend, projection change, actual scoring trend, risk	Divergence between market value and expected/actual production.
Manager behavior	Requires transactions and weekly lineup retention	starts, bench decisions, adds/drops, trades	Aggressiveness, lineup efficiency and roster churn.

11. Recommended Weekly Snapshot Extension

To support robust weekly trend reporting, preserve the current analysis rather than overwriting it. The recommended minimum historical model is below.

Table / collection	Required key	Minimum fields to retain	Reason
analysis_run	leagueId + generatedAt	appVersion, season, currentWeek, formatKey, sourceStatuses	Audit each run and distinguish data-source failures.
player_snapshot	leagueId + generatedAt + sleeperId	rosterId, rosterStatus, nflTeam, injuryStatus, depthOrder, actualPoints, actualPpg, projectedTotal, projectedGames, expectedPpg, dynastyValueRA, dynastyValueDP, dynastyValue, trend7d, trend30d, riskScore, riskTier, positionRank, positionTier	Enables player market, projection, risk, ownership and tier trends.
player_week	leagueId + season + week + sleeperId	rosterId, points, started, rostered, matchupId	Enables rolling production, start rate and waiver-impact reports.
team_snapshot	leagueId + generatedAt + rosterId	lineupPpg, depth, risk, totalValue, youngValue, currentPF, currentMaxPF, pickCapital, contenderScore, currentRank, franchiseScore, franchiseRank, component percentiles	Enables team movement and cause decomposition.
pick_snapshot	leagueId + generatedAt + season + round + originRosterId	ownerRosterId, previousOwnerRosterId, modelValue	Tracks pick movement and concentration.
transaction_fact	leagueId + transaction_id	created, week/leg, type, status, roster_ids, adds, drops, draft_picks, waiver_budget	Trade, waiver and roster-churn reporting.
source_snapshot	leagueId + generatedAt + source name	status, detail, endpoint/version	Separates true data movement from source outages.

11.1 Recommended Change Detection Fields

- $\text{delta_expected_ppg} = \text{current expectedPpg} - \text{prior snapshot expectedPpg}$
- $\text{delta_dynasty_value} = \text{current dynastyValue} - \text{prior snapshot dynastyValue}$
- $\text{delta_risk_score} = \text{current riskScore} - \text{prior snapshot riskScore}$
- $\text{delta_depth_order} = \text{prior depthOrder} - \text{current depthOrder}$ so positive values represent improvement
- $\text{ownership_changed} = \text{current rosterId differs from prior rosterId}$
- $\text{status_changed} / \text{injury_changed} / \text{tier_changed} = \text{categorical comparison to prior snapshot}$
- $\text{team_rank_delta} = \text{prior currentRank} - \text{current currentRank}$ so positive values represent a rise
- $\text{projection_share} = \text{player expectedPpg} / \text{team lineupPpg}$ or positional score
- $\text{value_efficiency} = \text{expectedPpg per 1,000 units of dynasty value}$
- $\text{market_vs_projection_divergence} = \text{percentile(dynasty value)} - \text{percentile(expectedPpg)}$

12. Data Quality, Limitations and Governance

Issue	Impact	Recommended control
Browser CORS availability	External APIs must allow browser requests. One source may fail while others succeed.	Persist sourceStatuses and avoid interpreting missing values as true zero.
Projection source differences	RosterAudit and Sleeper use different assumptions and field shapes.	Retain projectionSource and both raw source values where possible.
Expected PPG normalization	Dividing by 17 measures season availability but differs from active-game PPG.	Show projectedGames and sourcePpg beside expectedPpg in detailed reports.
Market-value scale differences	RosterAudit and DynastyProcess may use different scales/distributions.	A simple mean is currently used; future work should normalize percentiles or calibrate scales.
Missing IDP coverage	Public market-value coverage may be incomplete for defensive players.	Flag unsupportedSlots and exclude unsupported value metrics from rankings where appropriate.
Early-season sample size	Current PF is not used until Week 4.	Continue applying minimum-sample logic to player trends.
Weekly actual details not persisted	Current app aggregates matchup production and discards player-week rows after analysis.	Add player_week snapshot table.
Transactions not used in app analytics	The raw export has transactions, but current app does not retrieve/process them for reports.	Add Sleeper transactions endpoint and persistent transaction facts.
Pick model simplification	Team pick capital uses fixed values by round, not projected pick position.	Incorporate original-team rank/max PF and published pick values.
Name fallback matching	Some joins fall back to normalized player names.	Prefer Sleeper/GSIS IDs and track unmatched records.
Free agents excluded	Current player master is rostered players only.	Add a free-agent universe for waiver and breakout reports.
Snapshot retention	IndexedDB stores only the latest cached analysis per league ID.	Export or persist dated snapshots before overwriting.

13. Initial Weekly Player Trend Report Backlog

Report	Synopsis	Core data	Status
Market Momentum	Top 10 seven-day risers and fallers; team net value movement.	trend7d, dynastyValue, roster ownership	Current
Projection Pulse	Largest expected PPG and projected-games changes.	weekly player snapshots	Add snapshots
Role Risers	Depth-chart improvements, new starters and growing	depthOrder, starts, player-week rows	Add snapshots/player-week

Report	Synopsis	Core data	Status
	fantasy starts.		
Injury and Availability	New injuries, recoveries and projected-games changes.	injuryStatus, riskScore, projectedGames	Add snapshots
Breakout Radar	Young players gaining role, projection and scoring together.	age, years experience, depth, projections, weekly points	Add player-week; years_exp from supplemental pipeline
Buy Low / Sell High	Market trend diverging from projection and recent scoring.	market values, snapshots, rolling actual PPG	Add snapshots/player-week
Rookie Stock Watch	Rookie projection, depth, scoring and market movement.	draftYear, age, projection, depth, weekly scoring, values	Add snapshots/player-week
Team Opportunity Concentration	Players responsible for the largest share of team/position projection.	expectedPpg, positionScores, lineupPpg	Current
Waiver Wire Intelligence	Best available players and recently added/dropped assets.	full player universe, transactions, projections/value	Add free agents + transactions
Trade Market Review	Trades completed, value exchanged and projected lineup impact.	transactions, before/after snapshots, values/projections	Add transactions + snapshots
Dynasty Portfolio Report	Each manager's weekly value gain/loss by position and age cohort.	player snapshots, ownership, age, position	Add snapshots
Contender Movement	Power-ranking movers with component-level reasons.	team snapshots and score components	Add snapshots

Appendix A. Current Export Inventory

Export	Contents	Current filename pattern
Analysis JSON	Complete run-level object, teams, players, picks, methods and source statuses.	sleeper_{leagueId}_analysis.json
Rankings CSV	One row per team with ranking and summary metrics.	sleeper_{leagueId}_rankings.csv
Player Master CSV	One row per rostered player with identity, production, projection, value and risk.	sleeper_{leagueId}_players.csv
Report HTML	Application plus embedded completed analysis snapshot.	sleeper_{leagueId}_report.html
Print/PDF	Browser-rendered visual report.	User selected
Browser cache	Latest completed analysis keyed by league ID; daily player-directory cache.	IndexedDB SleeperDynastyAnalyzer

Appendix B. Glossary

Term	Definition
PF	Points for: points actually scored by a fantasy roster.
Max PF / potential points	Maximum possible points based on the source league roster and scoring.
Expected PPG	Projected season total divided by 17; includes availability effects.
Source PPG	Projection provider's raw PPG before 17-game normalization.
Dynasty value	Market-value estimate used to compare long-term fantasy assets.
Consensus value	Mean of available RosterAudit and DynastyProcess values.
Depth index	Weighted expected production of the top bench options.
Contender score	Composite current-season competitiveness measure.
Franchise score	Composite long-term dynasty health measure.
Snapshot	A dated preserved copy of fields used to calculate changes over time.
Grain	The level represented by one row or object.
Lineage	The source and transformations that produced a field.

9.14 Version 2.0 Player Tier Extension

Field	Type	Source / formula	Definition	Grain	Weekly trend use
tierPlayers	array	Derived from analysis.players	Relevant active QB/RB/WR/TE records eligible for the Player Tiers page. Includes rostered players and relevant free agents.	One collection per run	Base population for weekly tier snapshots.
longevityScore	number 0-100	Position age curve minus risk and status adjustments	Estimated career-runway score.	Player/run	Detect aging cliffs and durability changes.
projectionPct	number 0-100	Expected PPG percentile within position	Relative upcoming-season projection strength.	Player/run	Projection movement and positional rank.
valuePct	number 0-100	Dynasty value percentile within position	Relative market-supported value.	Player/run	Market movement and divergence reports.
longevityPct	number 0-100	Longevity-score percentile within position	Relative career-runway position.	Player/run	Career-longevity movement.
tierComposite	number 0-100	$0.60 \text{ projectionPct} + 0.25 \text{ valuePct} + 0.15 \text{ longevityPct}$	Overall application tier score.	Player/run	Week-over-week tier score delta.
analysisTier	string	Thresholds applied	S, 1, 2, 3, or 4	Player/run	Tier promotions

Field	Type	Source / formula	Definition	Grain	Weekly trend use
		to tierComposite	analytical tier.		and demotions.
rosterStatus = Free Agent	string	Derived when rosterId = 0	Relevant player not rostered in the analyzed Sleeper league.	Player/run	Waiver and free-agent watchlists.

10. Weekly Trend Reporting and Future Data Platform

The application can display a current snapshot immediately, but most week-over-week movement requires retaining dated historical records. The recommended architecture publishes one compact JSON snapshot per league and week, plus a current pointer file.

10.1 Recommended Weekly Snapshot Keys

- league_id + season + week + generated_at for run-level uniqueness.
- sleeper_id + snapshot_date for player-level history.
- roster_id + snapshot_date for team-level history.
- season + round + origin_roster_id + snapshot_date for future-pick ownership history.
- source_name + source_updated_at + schema_version for lineage and reproducibility.

10.2 Priority Weekly Reports

Report	Output
Tier Movers	Changes in analysisTier and tierComposite, with projection/value/longevity attribution.
Projection Risers and Fallers	Expected PPG and projected-total deltas.
Value versus Projection Divergence	Players whose market value and projected role move in opposite directions.
Waiver Tier Watch	Relevant free agents entering higher tiers or gaining projection/depth status.
Risk and Availability Movement	Injury, status, depth order, projected-games, and risk-score changes.
Team Power Movement	Contender rank, franchise rank, lineup PPG, depth, and draft-capital changes.
Draft Pick Movement	Ownership changes and concentration of future firsts.
Future nflverse Usage Trends	Targets, carries, routes, participation, air yards, red-zone usage, EPA, and rolling opportunity metrics.

11. Revision History

The following history incorporates the standalone revision-history document. Version 1.9 remains an in-development mobile initiative; Version 2.0 was released as the Player Tiers feature branch.

Document Purpose

This document records the major functional, analytical, visual, branding, and responsive-design changes made to the Fantasy Football Intelligence Hub from its initial single-file release through the latest validated desktop release. It also identifies the mobile-focused work that began for Version 1.9 but was not completed as a released build.

Release Overview

Version	Release	Status	Synopsis
1.0	Initial Release	Released	Reusable single-file Sleeper league analyzer with live API data, rankings, team analysis, exports, and saved reports.
1.1	League Selection and Team Refresh	Released	Introduced a dedicated league-selection landing page and explicit Update Team controls.
1.2	Unified Team State and Evidence	Released	Synchronized the selected team across tabs, added evidence-backed insights, player tiers, and a visual draft-capital dashboard.
1.3	Product Branding	Released	Renamed the application and introduced the logo and Honolulu Blue / Silver / Black / White visual identity.
1.4	Logo-Only Brand Header	Released	Removed header wording and enlarged the logo while retaining the sticky navigation below.
1.5	Expanded Brand Banner	Released	Cropped logo padding, enlarged and left-aligned the logo, added version control and creator attribution, and consolidated navigation into one row.
1.6	Navigation Sizing Correction	Released	Eliminated overlapping and truncation in the sticky navigation through explicit widths and horizontal-scroll fallback.
1.7	Header Control Relocation	Released	Moved focus-team and league controls into the main brand banner and added a created date to version control.
1.8	Layout Validation and Optimization	Released	Rebuilt header and navigation sizing, fixed malformed responsive CSS, and validated HTML, CSS, JavaScript, IDs, and breakpoints.

Version	Release	Status	Synopsis
1.9	Mobile Experience	In Development	Started a dedicated mobile layout for stacked header controls, swipeable tabs, full-width cards, and mobile-safe tables; no final released file was completed.

1.0 - Initial Release

Status: Released

Changes and Improvements

- Created a reusable, single-file HTML application capable of analyzing any public Sleeper NFL league by league ID.
- Added automatic league-format detection for 1QB versus superflex and PPR versus half-PPR settings.
- Parsed the league's actual starting roster positions and solved legal optimal lineups for every team.
- Integrated Sleeper league, roster, manager, matchup, player, and traded-pick data.
- Added projections, dynasty market values, risk scoring, bench-depth analysis, contender rankings, long-term franchise rankings, and team strategy recommendations.
- Included Dashboard, Team Insights, Optimal Lineups, Trade Finder, Player Master, Draft Capital, and Methodology sections.
- Added local browser caching and exports for JSON, rankings CSV, player CSV, saved HTML reports, and print/PDF output.

Release Synopsis

Established the complete analytical foundation and made the prior one-time report process reusable for any supported Sleeper league.

1.1 - League Selection and Team Refresh

Status: Released

Changes and Improvements

- Added a dedicated league-selection landing page as the application's initial screen.
- Automatically transitioned users from league setup to the main dashboard after a successful analysis.
- Added Change League and Refresh League Data controls.
- Introduced explicit Update Team buttons for the Dashboard, Team Insights, and Trade Finder.
- Replaced fragile dropdown change-event behavior with persistent selected-team state for dynamically rendered sections.

Release Synopsis

Improved the application flow and corrected the issue where changing a team did not reliably refresh the associated tables and reports.

1.2 - Unified Team State, Evidence, and Draft Capital

Status: Released

Changes and Improvements

- Consolidated team selection into one shared focus-team state used across all team-specific tabs.
- Added a global team selector and ensured Dashboard, Team Insights, Optimal Lineups, Trade Finder, Player Master, and Draft Capital followed the same selected roster.
- Expanded insights to include supporting players, projected PPG, positional ranks, dynasty values, risk information, and player tiers.
- Defined rostered-player tiers by same-position expected-PPG percentile.
- Redesigned Draft Capital with a visual capital leaderboard, team pick-portfolio cards, color-coded own and acquired picks, an ownership matrix, and acquired/sent pick movement detail.
- Added original-team and previous-owner information when available from Sleeper traded-pick records.

Release Synopsis

Made the analysis more transparent and actionable while turning draft capital into an at-a-glance strategic view instead of a simple ledger.

1.3 - Fantasy Football Intelligence Hub Branding

Status: Released

Changes and Improvements

- Renamed the product from Sleeper Dynasty Analyzer to Fantasy Football Intelligence Hub.
- Embedded the supplied Fantasy Football Intelligence Hub logo directly into the single HTML file.
- Introduced the product color palette: white background, Honolulu Blue, silver, black, and white.
- Moved the page selectors into a top banner treatment and placed the shared focus-team controls in the same persistent application area.
- Updated titles, footer language, and visible product references to the new brand.

Release Synopsis

Converted the analytical utility into a branded product with a consistent identity and navigation experience.

1.4 - Logo-Only Brand Header

Status: Released

Changes and Improvements

- Removed all wording from the upper brand header.
- Retained only the logo and increased its displayed size.
- Kept the navigation tabs, league information, team selector, refresh control, and change-league action in the sticky banner below.

Release Synopsis

Simplified the top brand area and gave the logo greater visual prominence.

1.5 - Expanded Banner and Integrated Navigation

Status: Released

Changes and Improvements

- Cropped transparent padding from the supplied logo so the visible mark filled the banner more effectively.
- Left-aligned the enlarged logo and sized it to nearly fill the banner height.
- Added a right-side Version Control panel with the current version and “Created by Mohamad Mokbel.”
- Reworked page selectors into a single non-wrapping row.
- Integrated tabs, focus-team selector, Update Team, Refresh, and Change League into one cohesive configuration/navigation strip.
- Added horizontal scrolling for narrower displays instead of allowing the navigation to wrap.

Release Synopsis

Improved brand presence and consolidated application controls into a more product-like command bar.

1.6 - Sticky Navigation Sizing Correction

Status: Released

Changes and Improvements

- Replaced compressed and overlapping navigation sizing with content-aware reserved widths.
- Removed ellipsis behavior from league name and metadata fields.
- Assigned explicit space to each tab, the team selector, Update Team, Refresh, and Change League controls.
- Kept all page selectors in one row and used whole-bar horizontal scrolling below the required width.
- Shortened only the displayed league metadata prefix while retaining the league ID, format, team count, and updated timestamp.

Release Synopsis

Corrected visible overlap, hidden text, and truncated controls in the sticky navigation.

1.7 - Focus Controls Moved to the Brand Banner

Status: Released

Changes and Improvements

- Moved the Focus Team selector from the sticky navigation into the main logo banner.
- Moved Update Team and Change League beside the selector between the logo and version panel.
- Simplified the sticky navigation to league information, page tabs, and Refresh Data only.
- Added “Created July 26, 2026” to the Version Control panel.
- Kept the application controls hidden until a league was loaded.
- Allowed the full header to scroll horizontally on narrower screens rather than overlap.

Release Synopsis

Separated global application configuration from page navigation while preserving one consistent focus-team control.

1.8 - Layout Optimization and Validation

Status: Released

Changes and Improvements

- Rebuilt the full brand-header and sticky-navigation sizing model with mathematically consistent container, column, gap, and padding dimensions.
- Corrected a malformed responsive CSS block that had left sizing rules outside the intended media query.
- Optimized panel, grid, KPI, filter, chart, and table sizing for desktop and laptop viewing.
- Ensured the logo column expands naturally without compressing the focus controls or version panel.
- Kept tables contained within their panels and added internal scrolling where required.
- Validated that the HTML contained no duplicate IDs.
- Validated that CSS parsed without errors and JavaScript passed syntax checking.
- Confirmed no horizontal scrolling was needed at common desktop widths, with controlled scrolling enabled for smaller screens.

Release Synopsis

Delivered the most stable and thoroughly validated desktop release, focused on reliable placement and viewing at common screen sizes.

1.9 - Dedicated Mobile Experience

Status: In Development

Changes and Improvements

- Began a dedicated mobile layout rather than simply shrinking the desktop interface.
- Planned a stacked mobile brand header with full-width focus-team controls and touch-friendly buttons.
- Planned horizontally swipeable page tabs with mobile scroll snapping.
- Planned full-width report cards, simplified multi-column layouts, and mobile-safe table scrolling.
- Confirmed that API calls can still work on mobile when the HTML is hosted over HTTP or HTTPS and the external services permit browser CORS requests.
- No finalized Version 1.9 HTML build was completed in the documented work session.

Release Synopsis

Mobile optimization is the next release area. A hosted HTTPS deployment is recommended for reliable mobile API access, especially for third-party data sources.

Release Management Notes

- Versions 1.0 through 1.8 were delivered as standalone HTML files, with each new version superseding the preceding file.
- Version 1.8 is the latest completed and validated desktop release documented here.
- Version 1.9 should be treated as planned/in-development until a finished HTML file is generated and validated.
- For mobile API use, deploy the HTML through HTTPS rather than opening it directly through a mobile file:// context.
- Future revision-history entries should record the release date, file name, status, summary, detailed changes, validation performed, and any known limitations.

11.10 Version 2.0 - Player Tiers and Expanded Relevant Player Pool

Status: Released

Changes and Improvements

- Added a dedicated Player Tiers page to the primary navigation.
- Expanded the analyzed player pool to include relevant active or market-supported free agents in addition to league-rostered players.
- Added filters for player name, position, analytical tier, roster status, and sort metric.
- Added expected PPG, dynasty value, career-longevity score, component percentiles, and a composite tier score.
- Established Tier S and Tiers 1 through 4 using an internally calculated model.
- Weighted tiers 60% toward upcoming-season projection, 25% toward dynasty value, and 15% toward position-adjusted career longevity.
- Added methodology documentation clarifying that application tiers are computed rather than copied from another website.
- Validated the release for duplicate IDs, CSS parse errors, and JavaScript syntax.

Release Synopsis

Introduced a league-independent player-ranking layer that prioritizes expected upcoming-season production while preserving dynasty value and career runway as secondary decision factors.

12. Release Maintenance Checklist

- ☐ Update APP_VERSION and visible version-control text in the HTML.
- ☐ Add the release entry and synopsis to this SDS revision history.
- ☐ Update the SDS application baseline and last-updated date.
- ☐ Add or revise architecture components if processing boundaries or integrations change.
- ☐ Add, modify, or deprecate data-dictionary fields and formulas.
- ☐ Update source status, endpoint, refresh, licensing, and limitation notes.
- ☐ Validate HTML IDs, CSS parsing, JavaScript syntax, API status, and responsive layout.
- ☐ Render and inspect this DOCX after every substantive documentation update.
- ☐ Release the HTML build and updated SDS together.